

Pages Router > ... > Routing > Pages and Layouts

Pages and Layouts

The Pages Router has a file-system based router built on the [concept of pages](#).

When a file is added to the `pages` directory, it's automatically available as a route.

In Next.js, a **page** is a [React Component](#) exported from a `.js`, `.jsx`, `.ts`, or `.tsx` file in the `pages` directory. Each page is associated with a route based on its file name.

Example: If you create `pages/about.js` that exports a React component like below, it will be accessible at `/about`.

```
1 export default function About() {  
2   return <div>About</div>;  
3 }
```

Index routes

The router will automatically route files named `index` to the root of the directory.

- `pages/index.js` → `/`
- `pages/blog/index.js` → `/blog`

Nested routes

The router supports nested files. If you create a nested folder structure, files will automatically be routed in the same way still.



- `pages/blog/first-post.js` → `/blog/first-post`

> `Menu` `pages/dashboard/settings/username.js` → `/dashboard/settings/username`

Pages with Dynamic Routes

Next.js supports pages with dynamic routes. For example, if you create a file called `pages/posts/[id].js`, then it will be accessible at `posts/1`, `posts/2`, etc.

To learn more about dynamic routing, check the [Dynamic Routing documentation](#).

Layout Pattern

The React model allows us to deconstruct a [page](#) into a series of components. Many of these components are often reused between pages. For example, you might have the same navigation bar and footer on every page.

`JS` components/layout.js



```
1 import Navbar from './navbar';
2 import Footer from './footer';
3
4 export default function Layout({ children }) {
5   return (
6     <>
7       <Navbar />
```

```
8      <main>{children}</main>
9      <Footer />
10    </>
11  );
12 }
```

Examples

Single Shared Layout with Custom App

If you only have one layout for your entire application, you can create a [Custom App](#) and wrap your application with the layout. Since the `<Layout />` component is re-used when changing pages, its component state will be preserved (e.g. input values).

JS pages/_app.js



```
1 import Layout from '../components/layout';
2
3 export default function MyApp({ Component, pageProps }) {
4   return (
5     <Layout>
6       <Component {...pageProps} />
7     </Layout>
8   );
9 }
```

Per-Page Layouts

If you need multiple layouts, you can add a property `getLayout` to your page, allowing you to return a React component for the layout. This allows you to define the layout on a *per-page basis*. Since we're returning a function, we can have complex nested layouts if desired.

JS pages/index.js



```
1
2 import Layout from '../components/layout'
3 import NestedLayout from '../components/nested-layout'
```

```
4
5 export default function Page() {
6   return (
7     /** Your content */
8   )
9 }
10
11 Page.getLayout = function getLayout(page) {
12   return (
13     <Layout>
14       <NestedLayout>{page}</NestedLayout>
15     </Layout>
16   )
17 }
```

JS pages/_app.js



```
1 export default function MyApp({ Component, pageProps }) {
2   // Use the layout defined at the page level, if available
3   const getLayout = Component.getLayout || ((page) => page);
4
5   return getLayout(<Component {...pageProps} />);
6 }
```

When navigating between pages, we want to *persist* page state (input values, scroll position, etc.) for a Single-Page Application (SPA) experience.

This layout pattern enables state persistence because the React component tree is maintained between page transitions. With the component tree, React can understand which elements have changed to preserve state.

Note: This process is called [reconciliation](#), which is how React understands which elements have changed.

With TypeScript

When using TypeScript, you must first create a new type for your pages which includes a `getLayout` function. Then, you must create a new type for your `AppProps` which overrides the `Component` property to use the previously created type.

TS pages/index.tsx

TypeScript ▾



```
1 import type { ReactElement } from 'react';
2 import Layout from '../components/layout';
3 import NestedLayout from '../components/nested-layout';
4 import type { NextPageWithLayout } from '../_app';
5
6 const Page: NextPageWithLayout = () => {
7   return <p>hello world</p>;
8 };
9
10 Page.getLayout = function getLayout(page: ReactElement) {
11   return (
12     <Layout>
13       <NestedLayout>{page}</NestedLayout>
14     </Layout>
15   );
16 };
17
18 export default Page;
```

TS pages/_app.tsx

TypeScript ▾



```
1 import type { ReactElement, ReactNode } from 'react';
2 import type { NextPage } from 'next';
3 import type { AppProps } from 'next/app';
4
5 export type NextPageWithLayout<P = {}, IP = P> = NextPage<P, IP> & {
6   getLayout?: (page: ReactElement) => ReactNode;
7 };
8
9 type AppPropsWithLayout = AppProps & {
10   Component: NextPageWithLayout;
11 };
12
13 export default function MyApp({ Component, pageProps }: AppPropsWithLayout) {
14   // Use the layout defined at the page level, if available
15   const getLayout = Component.getLayout ?? ((page) => page);
16
17   return getLayout(<Component {...pageProps} />);
18 }
```

Data Fetching

Inside your layout, you can fetch data on the client-side using `useEffect` or a library like [SWR](#). Because this file is not a [Page](#), you cannot use `getStaticProps` or `getServerSideProps` currently.

JS components/layout.js



```
1 import useSWR from 'swr';
2 import Navbar from './navbar';
3 import Footer from './footer';
4
5 export default function Layout({ children }) {
6   const { data, error } = useSWR('/api/navigation', fetcher);
7
8   if (error) return <div>Failed to load</div>;
9   if (!data) return <div>Loading...</div>;
10
11   return (
12     <>
13       <Navbar links={data.links} />
14       <main>{children}</main>
15       <Footer />
16     </>
17   );
18 }
```

[Previous](#)[< Routing](#)[Next](#)[Dynamic Routes >](#)

Was this helpful?



Resources

[Docs](#)[Learn](#)

More

[Commerce](#)[Contact Sales](#)

About Vercel

[Next.js + Vercel](#)[Open Source Software](#)

Legal

[Privacy Policy](#)[Cookie Preferences](#)

- Showcase

Blog

Analytics

Next.js Conf

Previews
- GitHub

Releases

Telemetry
- GitHub

Twitter

Subscribe to our newsletter

Stay updated on new releases and features, guides, and case studies.

you@domain.com

Subscribe

© 2023 Vercel, Inc.

